Lab 05 — Detecting MITM Attack on GPG Keys

Nenad Damjanović | Cybersecurity — FIŠ | Kali Linux / GnuPG

Practical laboratory report · 12–13 November 2026

Objective

Demonstrate why receiving a GPG public key does not automatically establish its owner's identity. Create separate local identities, substitute an attacker-controlled key, encrypt and decrypt a test message, identify the substitution through independent fingerprint verification, and explore the Web of Trust model.

Scope: Isolated educational simulation on one Kali Linux machine using separate GPG home directories. All names, addresses and messages are fictional. The report distinguishes the intended attack scenario in the instructor's slides from the behavior actually observed in the terminal.

1. Laboratory preparation

Created distinct Alice, Bobo and Mallory GPG homes with mode 700. The GnuPG version was 2.4.9. Added these directories to .gitignore to avoid publishing private keys. This separation simulates different people while the exercise runs on one computer.

```
mkdir -p alice_gpg bob_gpg mallory_gpg
chmod 700 alice_gpg bob_gpg mallory_gpg
gpg --version
ls -ld alice_gpg bob_gpg mallory_gpg
```

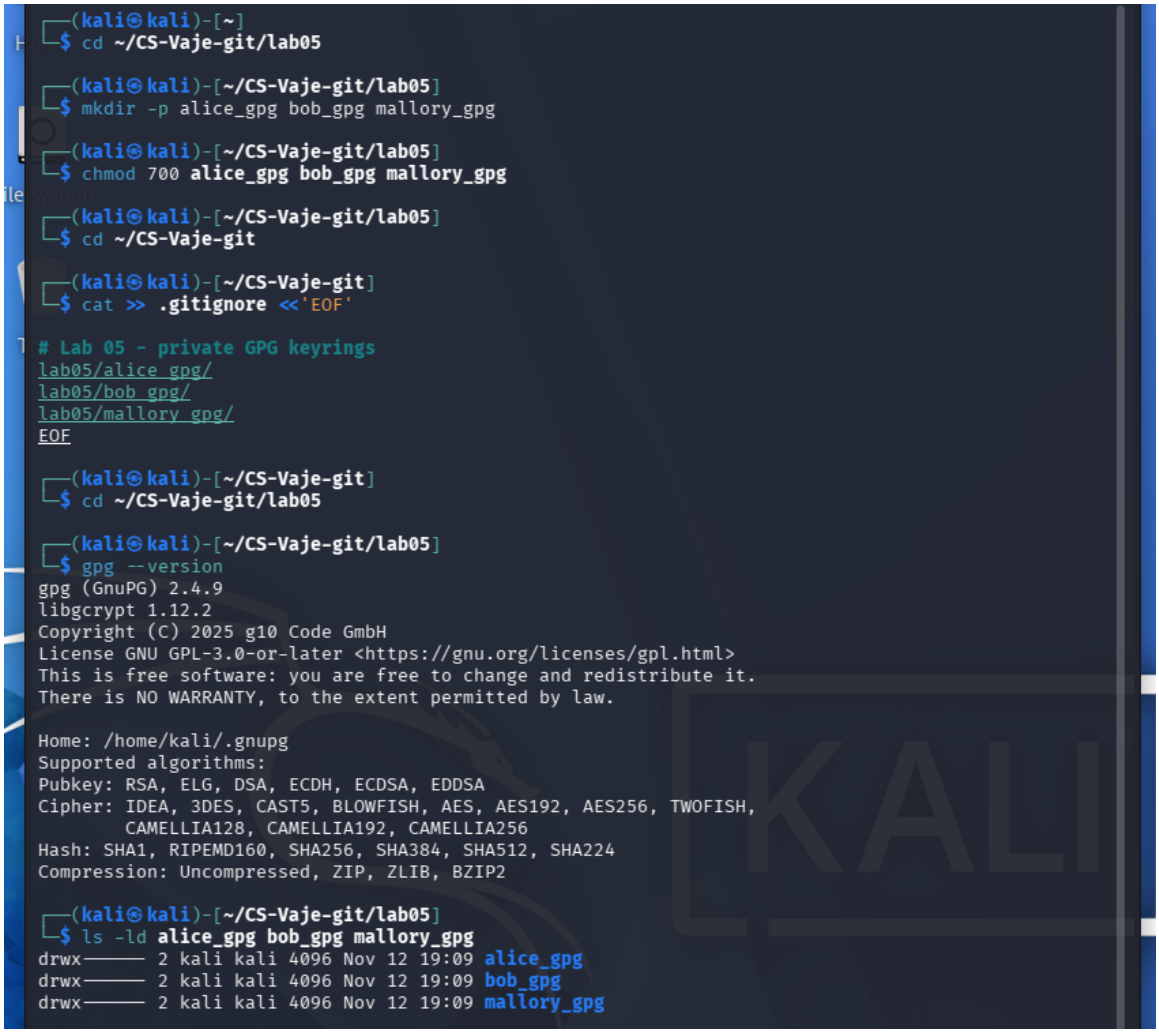


Figure 1. GPG setup, access permissions and Git exclusions.

2. Generating three identities

Alice and Bobo were generated with the instructor's interactive gpg --full-generate-key command, configured with RSA 4096-bit primary and encryption subkeys, expiring after one year. The actual recipient identity is Bobo <bobo@example.com>, not Bob <bob@example.com>. Mallory's key was generated with --quick-generate-key and uses Ed25519 for signing/certification and cv25519 for encryption. This deviation from the instructor's key-generation command is recorded rather than concealed.

```
gpg --homedir ./alice_gpg --full-generate-key
gpg --homedir ./bob_gpg --full-generate-key
gpg --homedir ./mallory_gpg --quick-generate-key "Mallory <mallory@example.com>" default default 1y
```

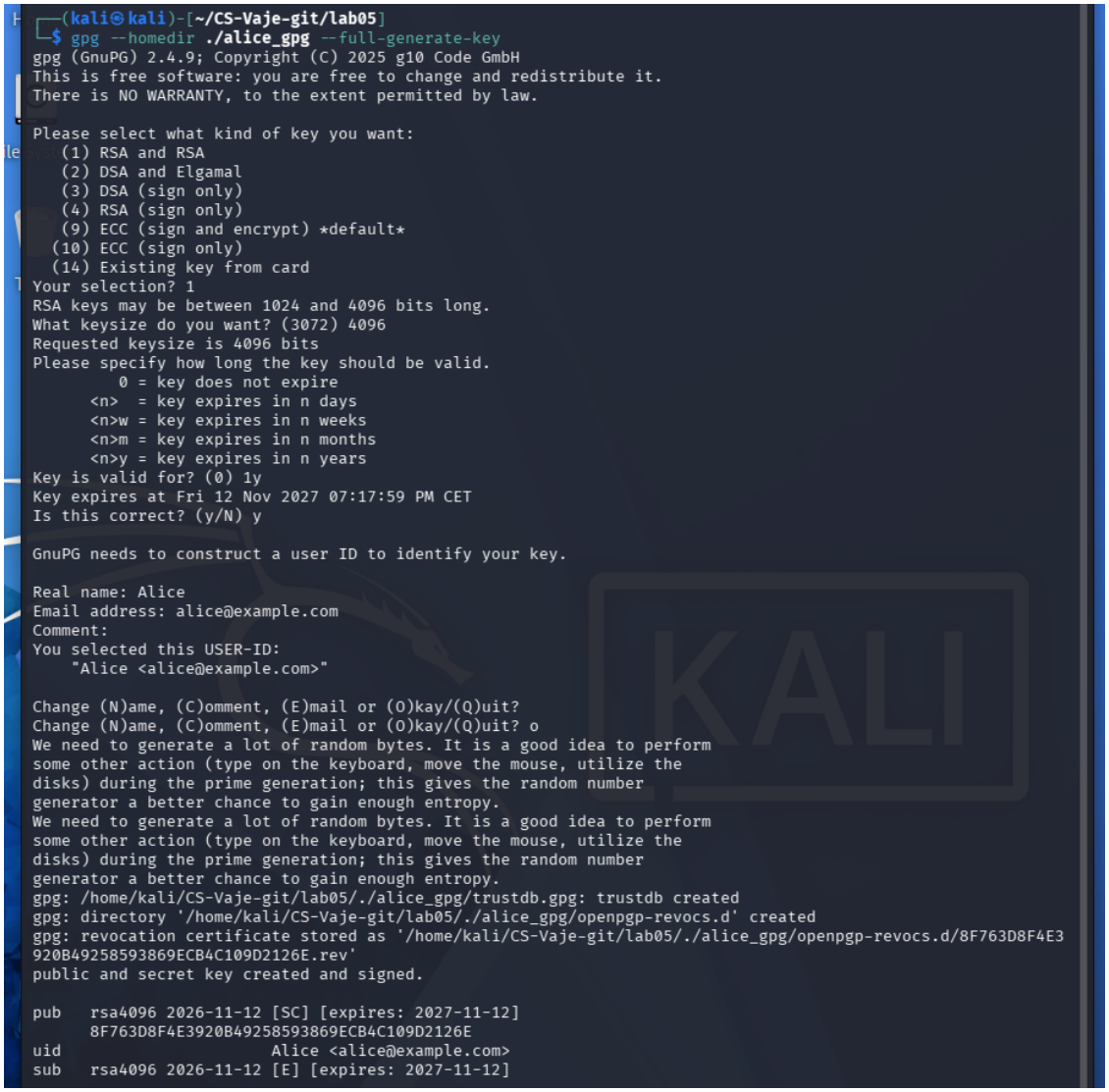


Figure 2. Alice: full interactive GPG key generation.

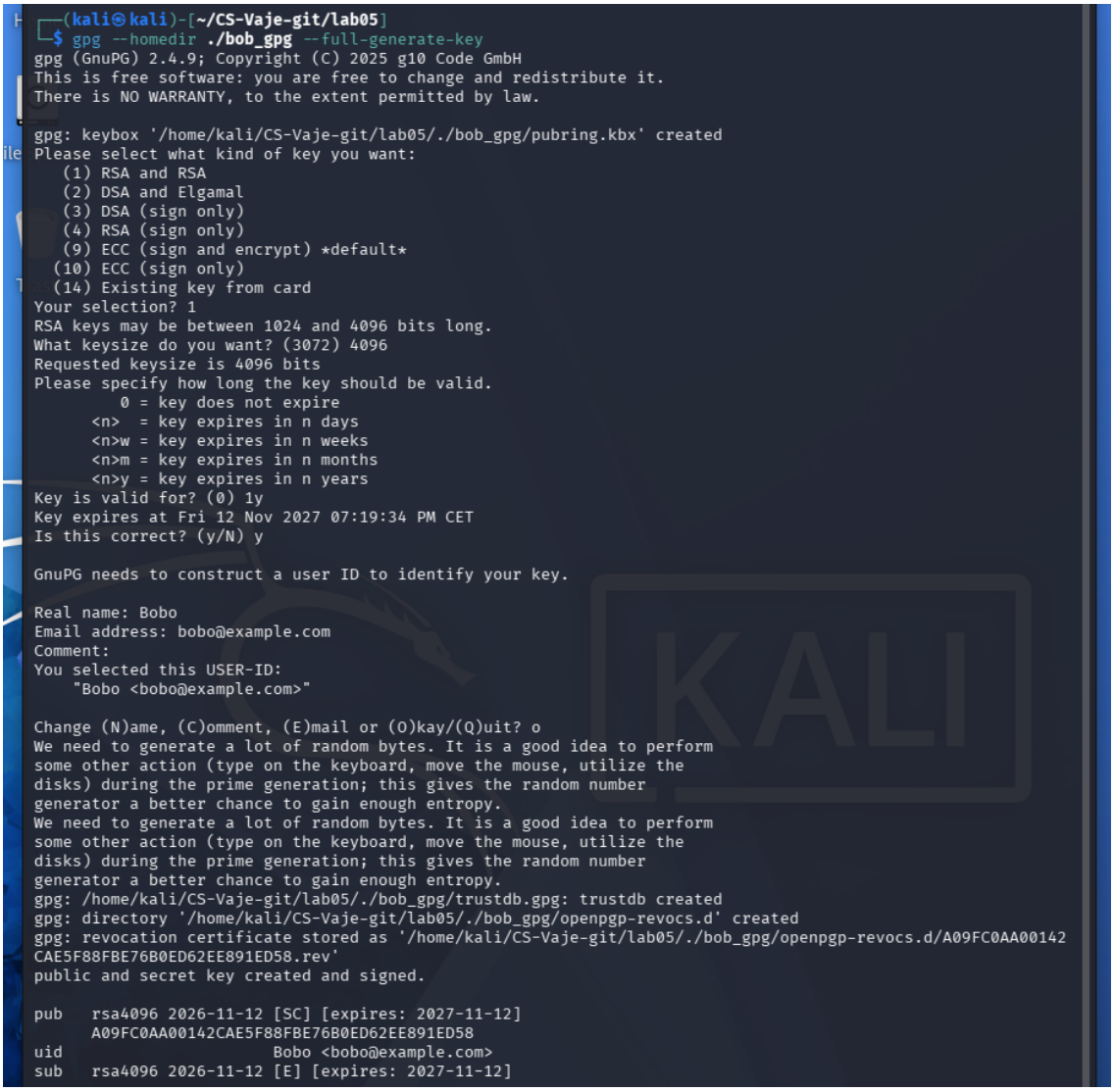


Figure 3. Bobo: full interactive GPG key generation.



Figure 4. Mallory: quick key generation (documented deviation).

3. Listing keys and comparing fingerprints

The --homedir parameter selects the intended user's separate keyring. Running gpg --fingerprint without --homedir searched the default ~/.gnupg directory and returned "No public key"; it did not indicate failure to generate the key. Each key was then listed and fingerprinted successfully within its own home.

```
gpg --homedir ./alice_gpg --fingerprint alice@example.com
gpg --homedir ./bob_gpg --fingerprint bobo@example.com
gpg --homedir ./mallory_gpg --fingerprint mallory@example.com
```

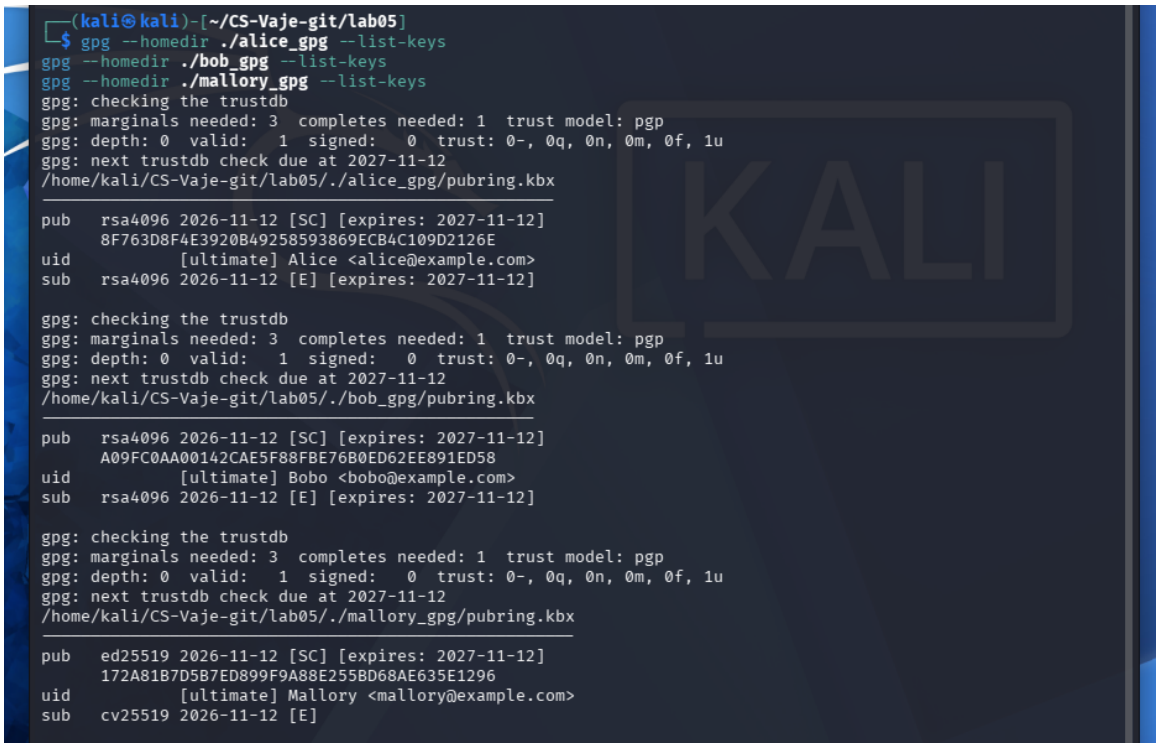


Figure 5. Distinct local keyrings and generated keys.

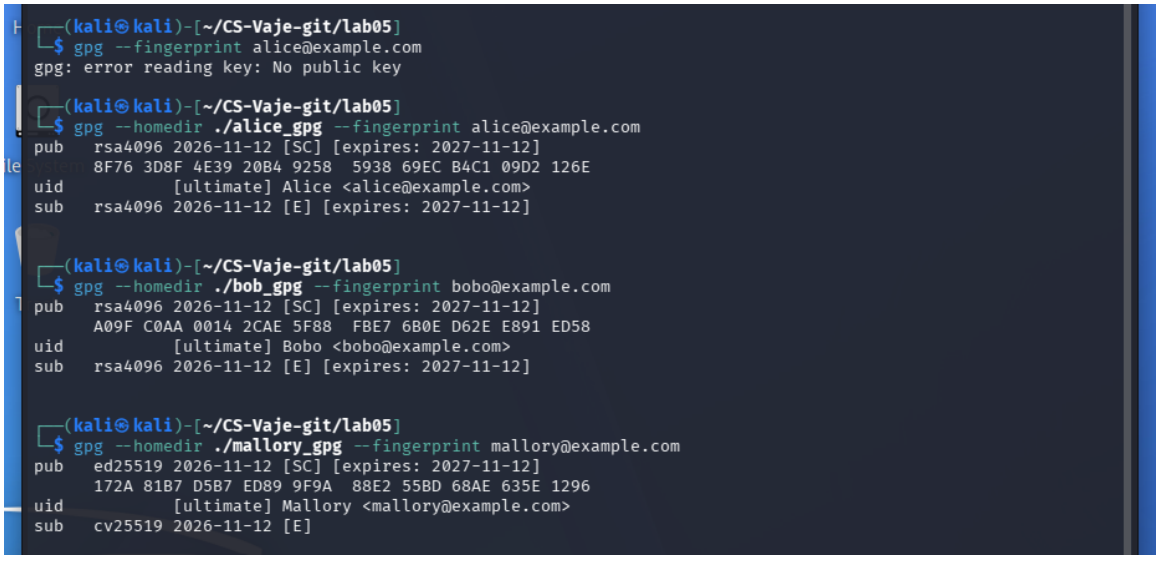


Figure 6. Fingerprints of Alice, Bobo and Mallory, including the initial default-keyring error.

4. MITM key substitution

Mallory exported his own public key but named the resulting file bobo_pubkey.asc, making it look like the recipient's key by its filename. Alice imported the file. The output shows that GPG still identified its user ID as Mallory <mallory@example.com>; renaming an .asc file does not rewrite or authenticate the public key. The substitution was therefore visible in the imported identity, rather than a fully convincing Bob impersonation.

```
gpg --homedir ./mallory_gpg --armor --export mallory@example.com > bobo_pubkey.asc
gpg --homedir ./alice_gpg --import bobo_pubkey.asc
gpg --homedir ./alice_gpg --list-keys
```

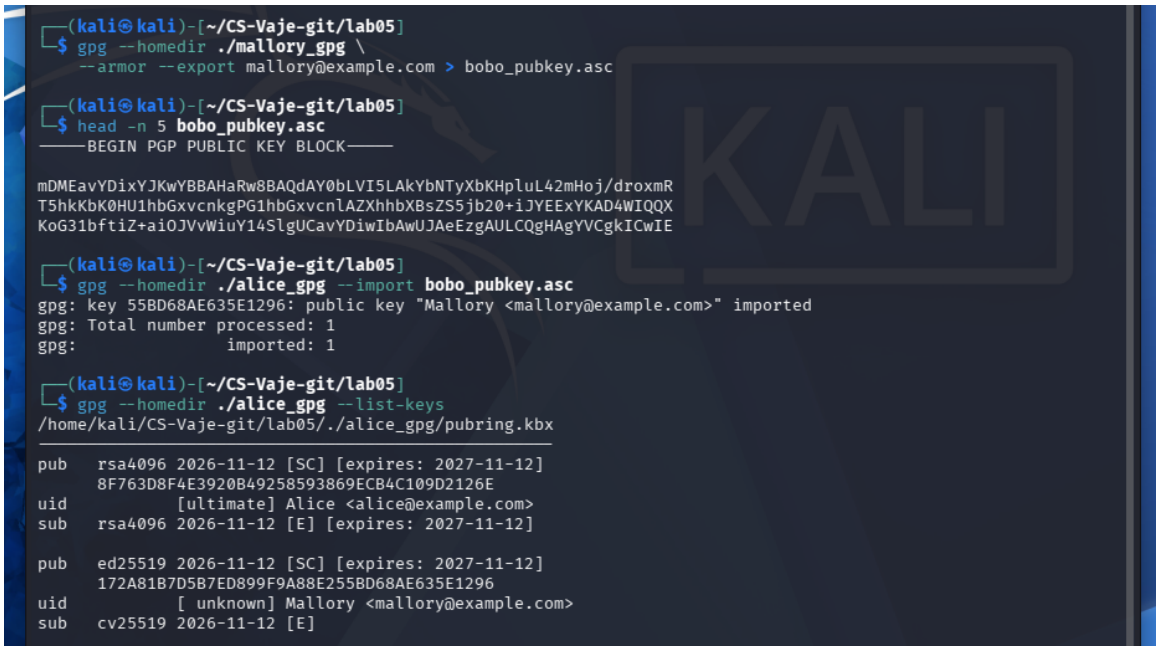


Figure 7. Attacker public key exported under a misleading filename and imported by Alice.

5. Encrypting the message: observed failure and controlled adaptation

Alice prepared the plaintext "Secret message for Bob". Encryption with the instructor's expected recipient name did not work: GPG attempted WKD lookup and returned "No data" because Alice did not have a key with that recipient identity. This is a substantive limitation in the slides' key-substitution example: exporting Mallory's key to a file called "Bob" does not make it match Bob's email. To demonstrate what happens after choosing an attacker-controlled key, Alice deliberately selected mallory@example.com; GPG warned that key ownership was not assured, and the test ciphertext secret.txt.gpg was produced.

```
echo "Secret message for Bob" > secret.txt
gpg --homedir ./alice_gpg --encrypt --recipient bobo@example.com secret.txt
gpg --homedir ./alice_gpg --encrypt --recipient mallory@example.com secret.txt
ls -lh secret.txt.gpg
```

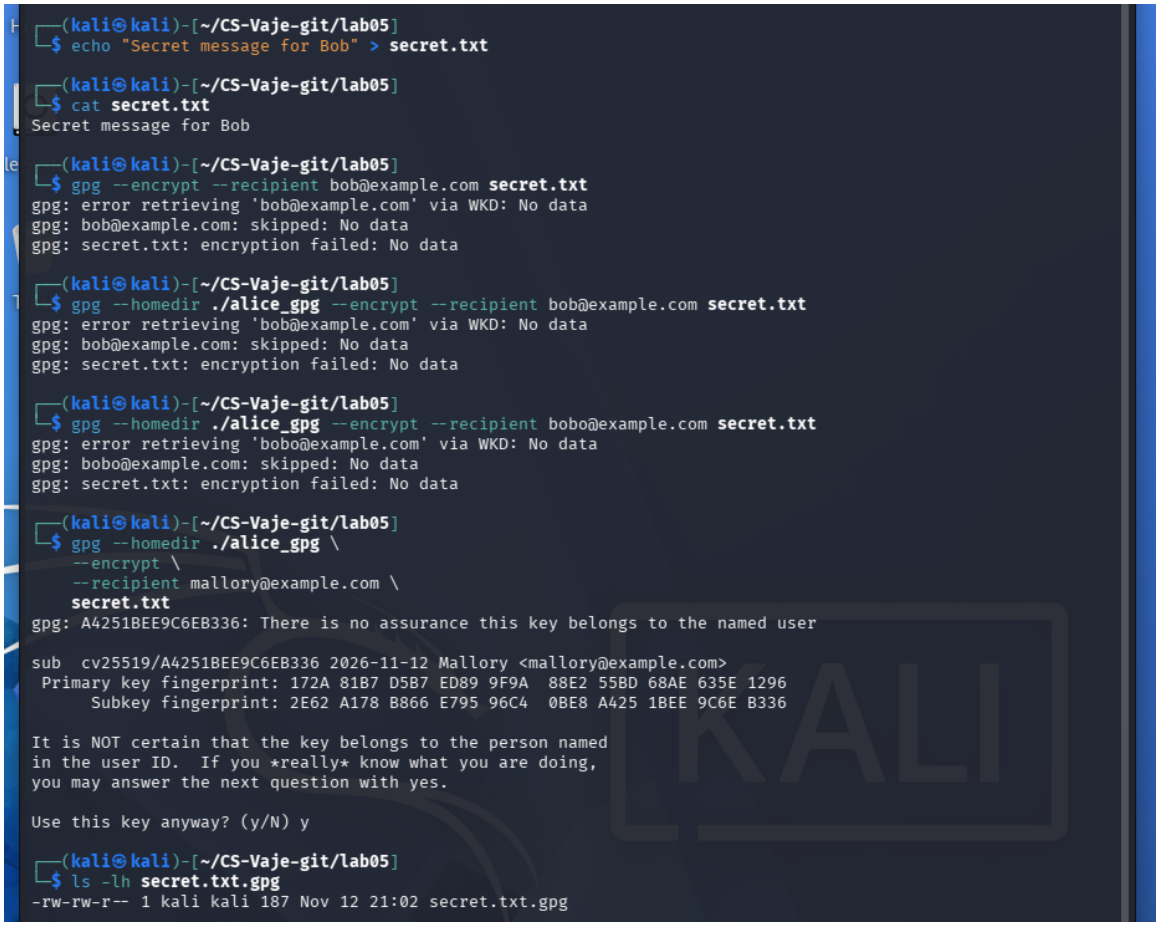


Figure 8. Failed encryption for Bobo and successful encryption to Mallory with a trust warning.

6. Mallory decrypts the ciphertext

Decrypting without the proper key in the default GPG home failed with “No secret key”. Running the operation against mallory_gpg succeeded and printed the exact message “Secret message for Bob”. This confirms the confidentiality failure if Alice encrypts for the wrong public key. It does not establish that the actual encryption to bobo@example.com succeeded; that operation demonstrably failed in the test.

```
gpg --decrypt secret.txt.gpg
gpg --homedir ./mallory_gpg --decrypt secret.txt.gpg
```

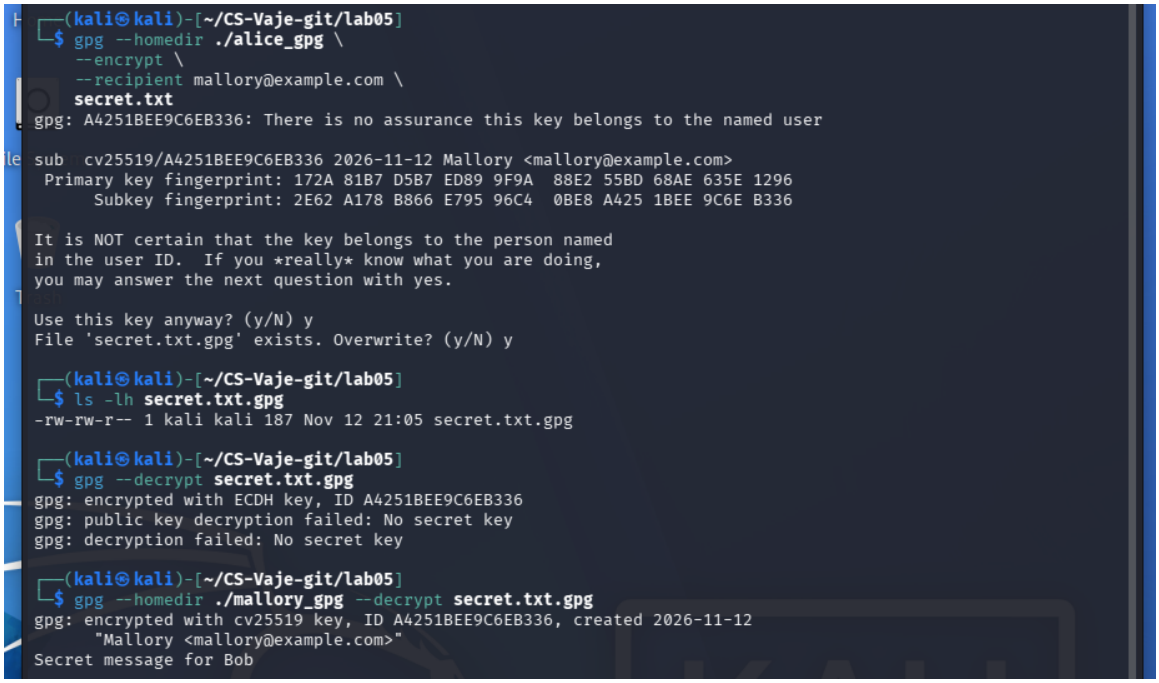


Figure 9. Only Mallory's corresponding private key successfully decrypts the test ciphertext.

7. Detecting the substituted key — detailed analysis

A fingerprint is a cryptographic digest derived from a public key and used as a compact identifier for comparison. It is public information, not a password or private key. A displayed GPG user ID (name and email) is supplied by the key creator; it is not independent proof of identity. Likewise, an exported filename such as bobo_pubkey.asc carries no cryptographic guarantee about its contents.

In the instructor's detection step, Bob shares his correct fingerprint with Alice through an independent trusted channel, such as an in-person meeting or a verified telephone call. Alice then computes the fingerprint of the imported key and compares the full values. If they differ, she must reject that key and obtain the authentic public key again. This protects against an adversary who controls the channel used to deliver the original key, provided the independent channel is not also compromised.

In this experiment, the authentic recipient (Bobo) had fingerprint A09F C0AA 0014 2CAE 5F88 FBE7 6B0E D62E E891 ED58. The substituted file bobo_pubkey.asc held Mallory's key, fingerprint 172A 81B7 D5B7 ED89 9F9A 88E2 55BD 68AE 635E 1296. These do not match. The mismatched user ID (Mallory versus Bobo) was a further, immediately visible red flag. At this point the safe decision is to reject the supplied key, not to encrypt confidential information to it.

The screenshots also show that Alice did not initially have Bobo's actual public key, so querying bobo@example.com returned "No public key". Later, Bobo's authentic key was exported as bob_real_pubkey.asc and imported into Alice's keyring; Alice and Bobo then displayed the same fingerprint. This is the correct recovery path after the suspicious key is detected. A fingerprint comparison proves correspondence to the key obtained through the independent channel; it does not establish identity if the supposed independent fingerprint source itself is fraudulent.

Key or exported file	Observed full fingerprint
Bobo (authentic)	A09F C0AA 0014 2CAE 5F88 FBE7 6B0E D62E E891 ED58
Mallory (substituted)	172A 81B7 D5B7 ED89 9F9A 88E2 55BD 68AE 635E 1296

gpg --homedir ./bob_gpg --fingerprint bobo@example.com
gpg --homedir ./alice_gpg --fingerprint mallory@example.com
gpg --show-keys --fingerprint bobo_pubkey.asc
gpg --show-keys --fingerprint bob_real_pubkey.asc

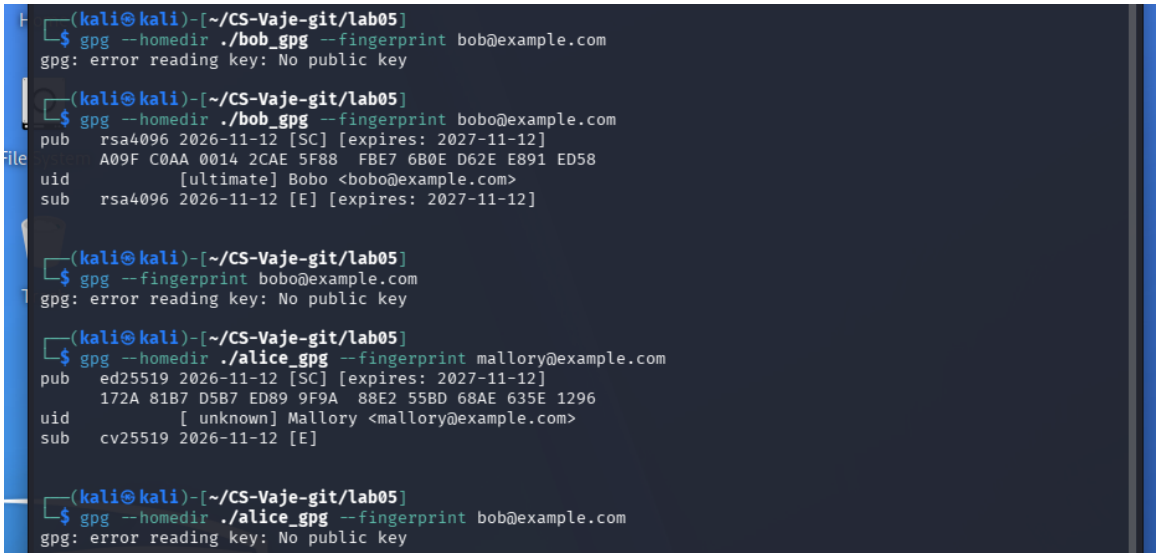


Figure 10. Fingerprint mismatch and absence of Bobo's public key in Alice's initial keyring.

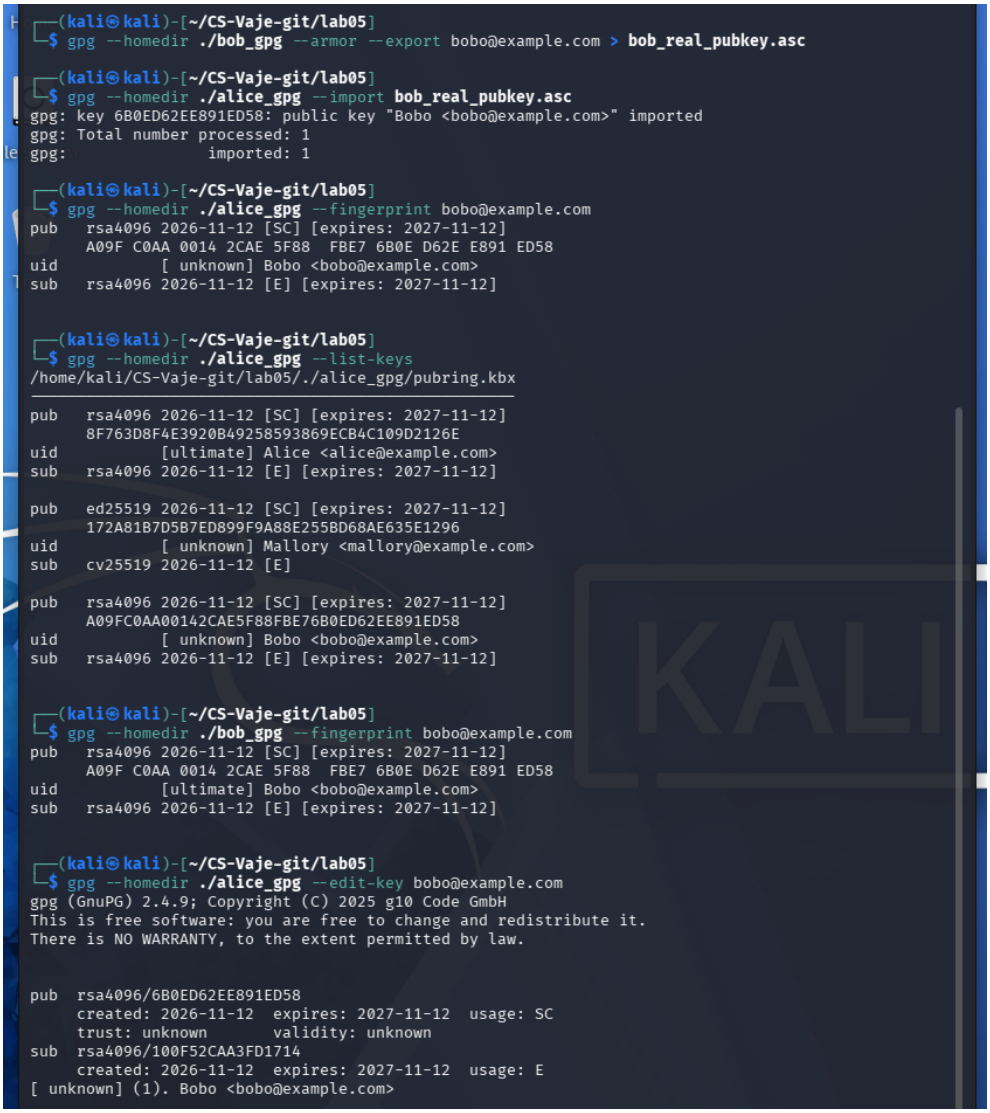


Figure 11. Authentic Bobo public key imported; both keyrings show the same fingerprint.

8. Web of Trust and ownertrust

The instructor's final exercise uses gpg --edit-key followed by trust, 5, quit. The recording shows that Alice selected ownertrust 5 (ultimate) for Bobo and confirmed it. This demonstrates how to set the setting in GPG, but ultimate ownertrust should normally be reserved for keys under the user's own control. Merely matching a fingerprint does not justify granting someone full or ultimate authority to certify third-party identities.

GPG distinguishes key validity (confidence a public key belongs to its claimed user ID) from ownertrust (confidence that its owner correctly verifies and certifies other people's keys). The displayed output initially showed Bobo's validity as unknown even though the imported key was technically usable. Trusting a user and cryptographically certifying their key are distinct actions; a trust choice alone should not be described as a completed identity verification.

```
gpg --homedir ./alice_gpg --edit-key bobo@example.com
gpg> trust
Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y
gpg> quit
```

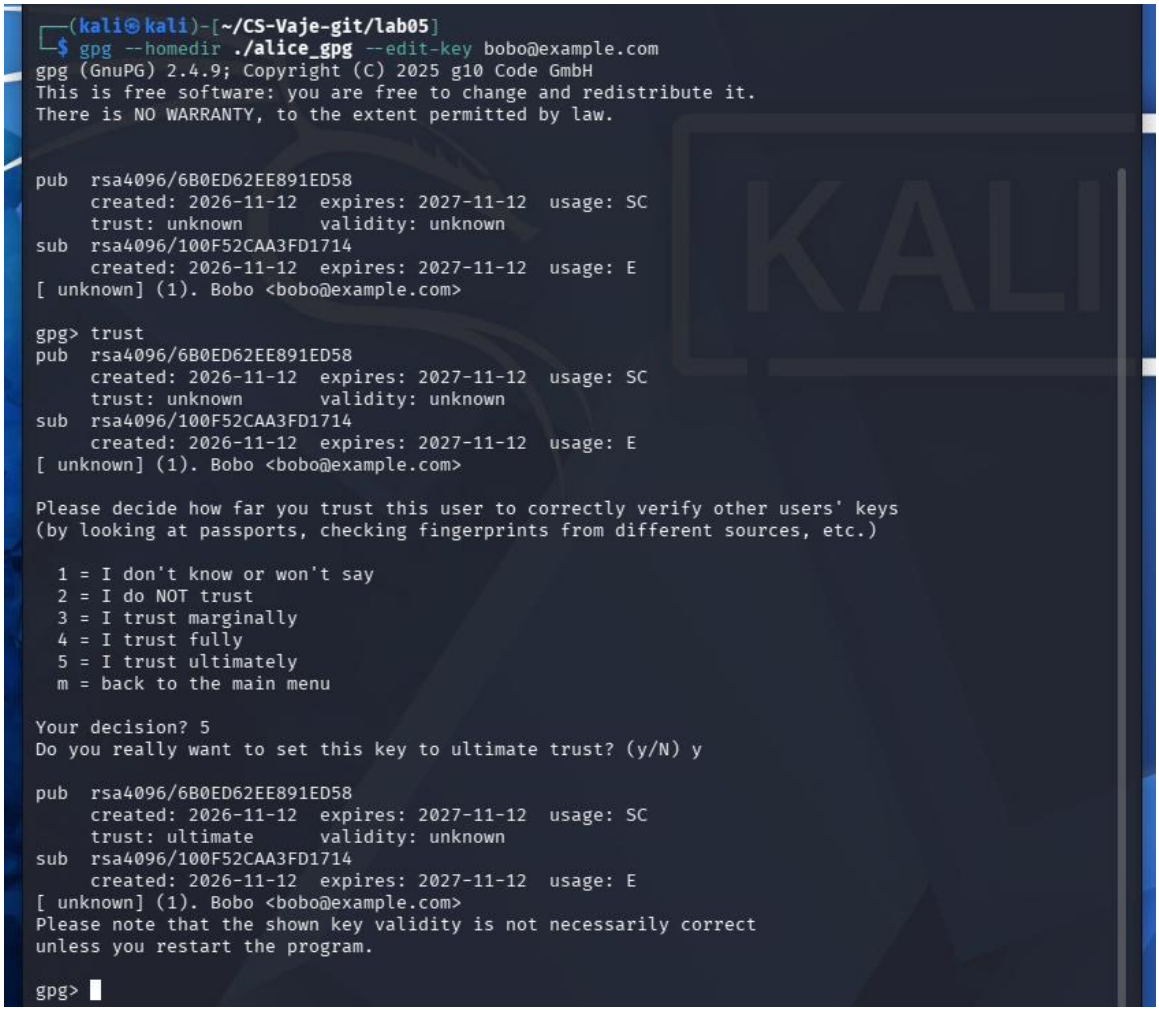


Figure 12. Instructor's Web of Trust command and recorded selection of ultimate ownertrust.

9. Findings and security recommendations

Findings: (1) A filename and a GPG user ID are not proof of authenticity. (2) GPG rejected attempts to encrypt to an unknown Bobo key, rather than silently treating the substituted Mallory file as Bobo's key. (3) When encryption explicitly targeted Mallory's key, only Mallory's private key could recover the test message. (4) Independent fingerprint comparison exposed the mismatch, and importing the genuine key restored the correct identity association. (5) GPG ownertrust levels must not be confused with verifying identity.

Recommendations: verify complete public-key fingerprints via an authenticated independent channel before first use; reject mismatches and investigate key substitutions; pay attention to warnings about unknown key validity; protect private keys with strong passphrases and restrictive permissions; never commit private GPG homes or actual secrets; reserve ultimate ownertrust for the user's own keys; and maintain a documented procedure for key rotation and revocation.

Conclusion

This laboratory demonstrated the core danger of GPG key substitution and the value of fingerprint verification. The observed attempt to encrypt to Bobo failed because the imported key was still identified as Mallory; a separate, explicitly targeted encryption to Mallory was decrypted successfully by the attacker keyring. The exercise also confirmed that the real Bobo key can be distinguished by comparing full fingerprints and by

importing the authentic public key. These observations illustrate why confidentiality depends on verifying the recipient key, not merely on using a strong encryption algorithm.

ChatGPT is used to create a report from my own work and screenshots